Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed
1130 / 1130

Attempts : Correct / Total
1 / 5
Accuracy : 20%

Points Scored
1 / 1
Your Total Score: 7

Time Taken
0.01

C++ (17)
Start Timer

```cpp
class Solution {
  public:
  // out state is just what number am i on on the factoial series
    int factorial(int n) {
        // edge case is 0! = 1
        if(n == 0){
            return 1;
        }
        // recurnce ralation is n! = n * (n-1)!
        return n * factorial(n-1);

    }
};
```

Custom Input | Compile & Run | Submit

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 31 / 31 testcases passed

yousab21 submitted at Jul 20, 2026 14:05

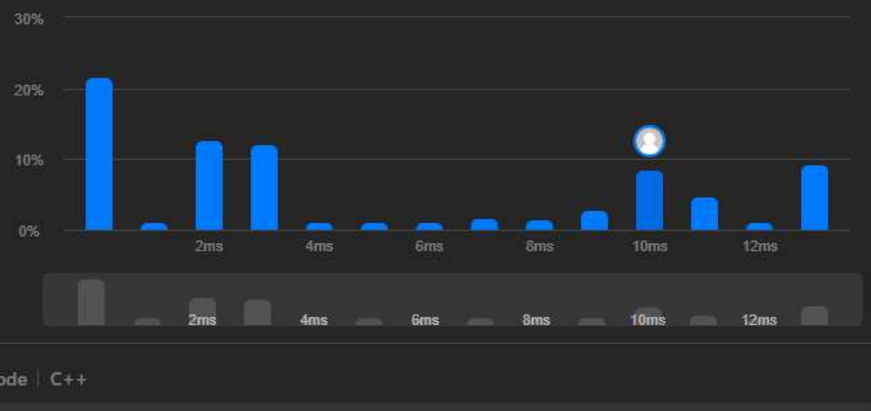
Analysis | Solution

Runtime

10 ms | Beats 46.41%

Memory

7.76 MB | Beats 65.38%



Code | C++

```cpp
class Solution {
public:
// state : number we are on in the series
    int fib(int n) {
        // basecase : fib(0) = 0, fib(1) = 1;
       if(n==0 || n ==1){
        return n;
       }
```

View more

More challenges

- 70. Climbing Stairs
- 873. Length of Longest Fibonacci Subsequence
- 1137. N-th Tribonacci Number

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```cpp
class Solution {
public:
// state : number we are on in the series
    int fib(int n) {
        // basecase : fib(0) = 0, fib(1) = 1;
       if(n==0 || n ==1){
        return n;
       }
        //recurnce relation : fib(n) = fib(n-1) + fib(n-2)
       return fib(n-1) + fib(n-2);
    }
};
```

--INSERT--

Saved Ln 12, Col 3

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 55 / 55 testcases passed

yousab21 submitted at Jul 20, 2026 14:43

Analysis | Solution

Runtime

0 ms | Beats 100.00%

Memory

11.22 MB | Beats 20.37%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 25. Reverse Nodes in k-Group
- 1721. Swapping Nodes in a Linked List

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        // keep going down the array
        if(!head || !head->next){
            return head;
        }

        // if we assume swapPairs to be the black magic function that always returns the
        // head of corectly swapped list from a given node to the end of the list
        // then if we give it the start the next pair we can expect it to return it swapped
        // so we can just rewire it head->next->next to it
        head->next->next = swapPairs(head->next->next);

        // here we need to eventually return the head of the entire list
        // after rewiring we we need to start swapping from the end of the list
        // and walk out way to the start, swapping the pairs on our way back
        ListNode* current = head->next;
        head->next = current->next;
        current->next = head;
        head= current;

        // once we are done recuring we now have head to on the first node of the swapped list
        return head;
    }
};
```

--NORMAL--

Saved Ln 13, Col 1

Testcase | Test Result

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
160 / 160	**1 / 1**
	Accuracy : 100%
Points Scored	Time Taken
2 / 2	**0.02**
Your Total Score: 9	

C++ (17) | Start Timer

```cpp
class Solution {
  public:
    void printNos(int n) {
        // we stop  recurring at 0
        if(n == 0 ){
            return;
        }
        // we print out cuurent n
        cout<< n << " ";
        // we go to the next number (n - 1)
        printNos(n - 1);
    }
};
```

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 4**
	Accuracy : 25%

Points Scored	Time Taken
4 / 4	**0.06**
Your Total Score: 13 ↑	

C++ (17) Start Timer

```cpp
class Solution {
  public:
    string removeUtil(string &s) {

        int start = 0;
        int end = 1;
        bool dupsFound = false;
        bool stringChanged = false;


        // scan the string from left the right for duplicates
        while(start < s.size() - 1 ){

            end = start + 1;

            // if there is a duplicate chain find the full range
            while(end < s.size() && s[start] == s[end]){
                end++;
                dupsFound = true;
            }

            // if we found a dup chain remove it
            if(dupsFound){
                s.erase(start, end-start);
                dupsFound = false;
                stringChanged = true;
            }
            //if we remove duplicates the shifted substring be now begin a duplicate chain
            // so dont move start and check again

            // if the char we are on is not part of a dup chain move start to the right
            else{
                start++;
            }

        }
        // base case: we scanned the whole string and there is no more dups
        if(!stringChanged){
            return s;
        }

        // if we did changes on the string that may have created antother dups so we scan again
        // we modify the string by refrence so we can just trow away the outputs of the recursive calls
        // other than than the base case
        else{
            return removeUtil(s);
        }

    }

};
```

Custom Input | Compile & Run | Submit

Problem List

Submit

Premium

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 34 / 34 testcases passed

yousab21 submitted at Jul 21, 2026 18:19

Analysis | Solution

Runtime

0 ms | Beats 100.00%

Memory

9.56 MB | Beats 34.93%

Code | C++

```
1 // i had to look up some parts of this problem to be honest. which is a good
2 // my solution was originally 134 lines, turns out the stl already has funct
3 class Solution {
4 public:
5     string decodeString(string s) {
6
7         // we find the first closing braket this will be the end of the righ
8         int close = s.find(']');
```

View more

More challenges

- 471. Encode String with Shortest Length
- 726. Number of Atoms
- 1087. Brace Expansion

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
1  // i had to look up some parts of this problem to be honest. which is a good thing because
2  // my solution was originally 134 lines, turns out the stl already has functions for most of the logic i was doing
3  class Solution {
4  public:
5      string decodeString(string s) {
6
7          // we find the first closing braket this will be the end of the rightmost deepest query
8          int close = s.find(']');
9
10         // out base case it if find returned npos which means there is no querys left
11         if (close == string::npos){
12             return s;
13         }
14
15         // we now also Find matching '[' by scannig left from where we found the closing braket
16         int open = close;
17         while (s[open] != '['){
18             open--;
19         }
20         // we now have the indexes of the start and end of the string we need to repeat
21
22         // now we need to find the numerical "repeat this many times" value
23         // we start scanning from just right of the braket
24         int numStart = open - 1;
25         // there may be more than one digit so we need to scan until we get all the digits
26         while (numStart >= 0 && isdigit(s[numStart])){
27             numStart--;
28         }
29         numStart++;
30         // once we get all the digits we can convert the sub string into an int
31         int k = stoi(s.substr(numStart, open - numStart));
32
33         // the "inside" string is just substring betweenthe brakets
34         string inside = s.substr(open + 1, close - open -1);
35
36         // we now can do the actual decoding
37         string decoded;
38         for (int i = 0; i < k; i++){
39             decoded += inside;
40         }
41
42         // we now replace the whole "k[inside]" substring with its decded version
43         s.replace(numStart, close - numStart + 1, decoded);
```

--INSERT--

Saved

Ln 2, Col 85

Testcase | Test Result

Problem List

Submit

Premium

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 34 / 34 testcases passed

yousab21 submitted at Jul 21, 2026 18:19

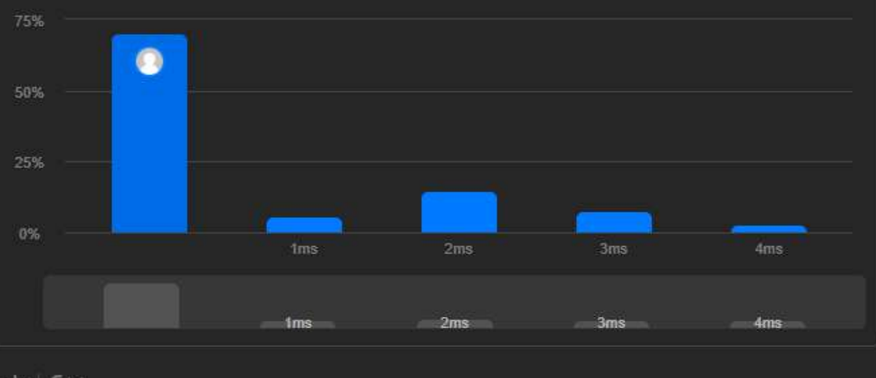
Analysis | Solution

Runtime

0 ms | Beats 100.00%

Memory

9.56 MB | Beats 34.93%



Code | C++

```
// i had to look up some parts of this problem to be honest. which is a good
// my solution was originally 134 lines, turns out the stl already has funct
class Solution {
public:
    string decodeString(string s) {

        // we find the first closing braket this will be the end of the righ
        int close = s.find(']');
```

View more

More challenges

- 471. Encode String with Shortest Length
- 726. Number of Atoms
- 1087. Brace Expansion

Write your notes here

Select related tags 0/5

Code Submit Ctrl Enter

C++ Auto

```
    if (close == string::npos){
        return s;
    }

    // we now also Find matching '[' by scannig left from where we found the closing braket
    int open = close;
    while (s[open] != '['){
        open--;
    }
    // we now have the indexes of the start and end of the string we need to repeat

    // now we need to find the numerical "repeat this many times" value
    // we start scanning from just right of the braket
    int numStart = open - 1;
    // there may be more than one digit so we need to scan until we get all the digits
    while (numStart >= 0 && isdigit(s[numStart])){
        numStart--;
    }
    numStart++;
    // once we get all the digits we can convert the sub string into an int
    int k = stoi(s.substr(numStart, open - numStart));

    // the "inside" string is just substring betweenthe brakets
    string inside = s.substr(open + 1, close - open -1);

    // we now can do the actual decoding
    string decoded;
    for (int i = 0; i < k; i++){
        decoded += inside;
    }

    // we now replace the whole "k[inside]" substring with its decded version
    s.replace(numStart, close - numStart + 1, decoded);

    // continue decoding the rest of the string
    // as we also need the fullyt decoded strings we dont need the partially decoded
    // version that get returned during recursive calls
    return decodeString(s);
}
};
```

--INSERT--

Saved Ln 2, Col 85

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 93 / 93 testcases passed

yousab21 submitted at Jul 21, 2026 19:26

Analysis | Solution

Runtime

4 ms | Beats 49.43%

Memory

121.95 MB | Beats 39.95%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 2130. Maximum Twin Sum of a Linked List

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
*     ListNode(int x, ListNode *next) : val(x), next(next) {}
* };
*/

// i couldnot find the recursive method to do this insted i reversed the second half of the list and
// scanned with 2 pointers
class Solution {
    ListNode* reverseList(ListNode* head){
        ListNode* prev = nullptr;
        ListNode* current = head;
        ListNode* nextNode = current->next;

        while(current){
            nextNode = current->next;
            current->next = prev;
            prev = current;
            current = nextNode;
        }
        return prev;
    }
public:
    bool isPalindrome(ListNode* head) {
        if(!head->next){
            return true;
        }

        // use fast and slow method to get thte middle of the list
        ListNode* fast = head;
        ListNode* slow = head;
        while(fast->next && fast->next->next){
            slow = slow->next;
            fast = fast->next->next;
        }
        // now slow sits right bofore the middle of the list this is done by stoping fast before the
        // last usual operation

        // we can now reverse the sublist and rewire is so we dont take extra space
        slow->next = reverseList(slow->next);

        // now we can walk the 2 pointers and compare
        ListNode* right = head;
        ListNode* left  = slow->next;

        while(left){
```

--INSERT--

Saved Ln 58, Col 9

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 93 / 93 testcases passed

yousab21 submitted at Jul 21, 2026 19:26

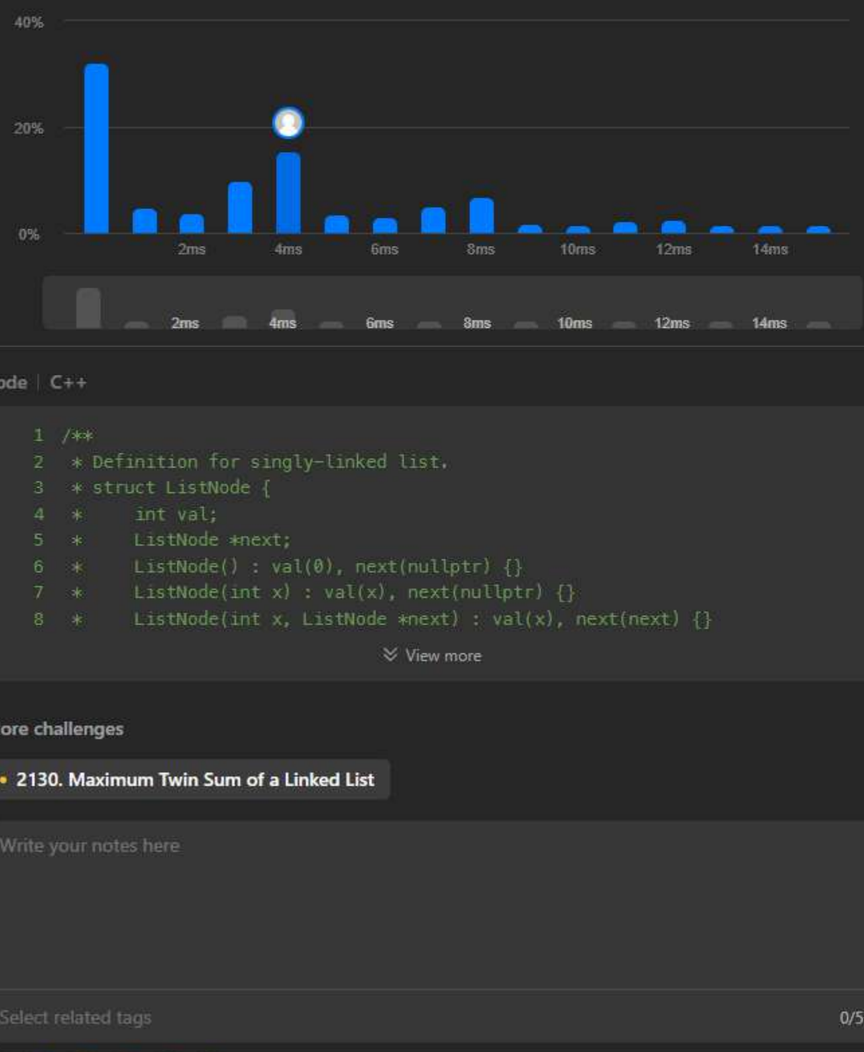
Analysis | Solution

Runtime

4 ms | Beats 49.43%

Memory

121.95 MB | Beats 39.95%



Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 2130. Maximum Twin Sum of a Linked List

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
22              current->next = prev;
23              prev = current;
24              current = nextNode;
25          }
26          return prev;
27      }
28  public:
29      bool isPalindrome(ListNode* head) {
30          if(!head->next){
31              return true;
32          }
33
34          // use fast and slow method to get thte middle of the list
35          ListNode* fast = head;
36          ListNode* slow = head;
37          while(fast->next && fast->next->next){
38              slow = slow->next;
39              fast = fast->next->next;
40          }
41          // now slow sits right bofore the middle of the list this is done by stoping fast before the
42          // last usual operation
43
44          // we can now reverse the sublist and rewire is so we dont take extra space
45          slow->next = reverseList(slow->next);
46
47          // now we can walk the 2 pointers and compare
48          ListNode* right = head;
49          ListNode* left  = slow->next;
50
51          while(left){
52              if(left->val != right->val){
53                  return false;
54              }
55              left= left->next;
56              right = right->next;
57          }
58
59          return true;
60      }
61  };
```

--INSERT--

Saved Ln 58, Col 9

Testcase | Test Result